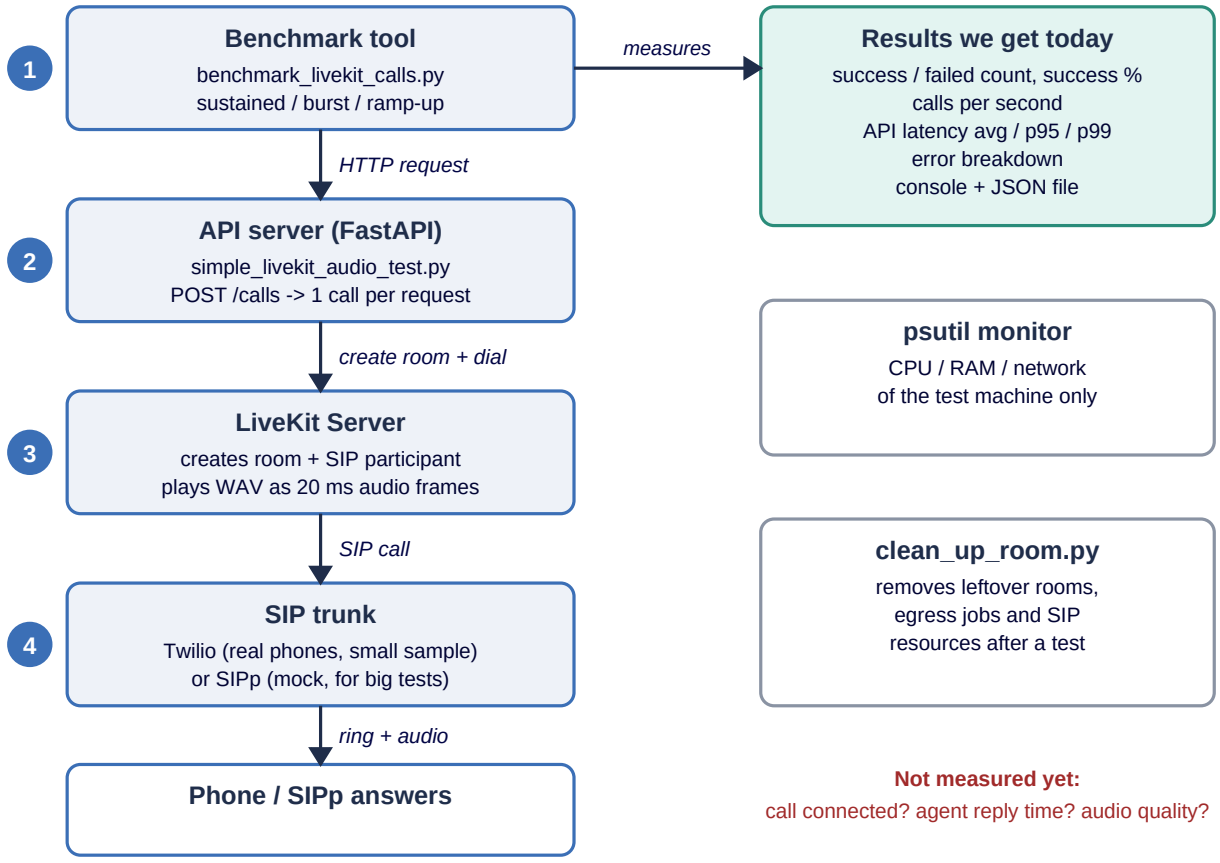
LiveKit Load-Test Framework

Two diagrams: what we have today, and where we are going

How to read this: Diagram 1 is the framework that exists in our repo now. Diagram 2 is the target setup from the team's whiteboard discussion. The gap between them is our work plan. No load test has been run yet, so capacity numbers here are estimates.

Diagram 1 - Our framework today

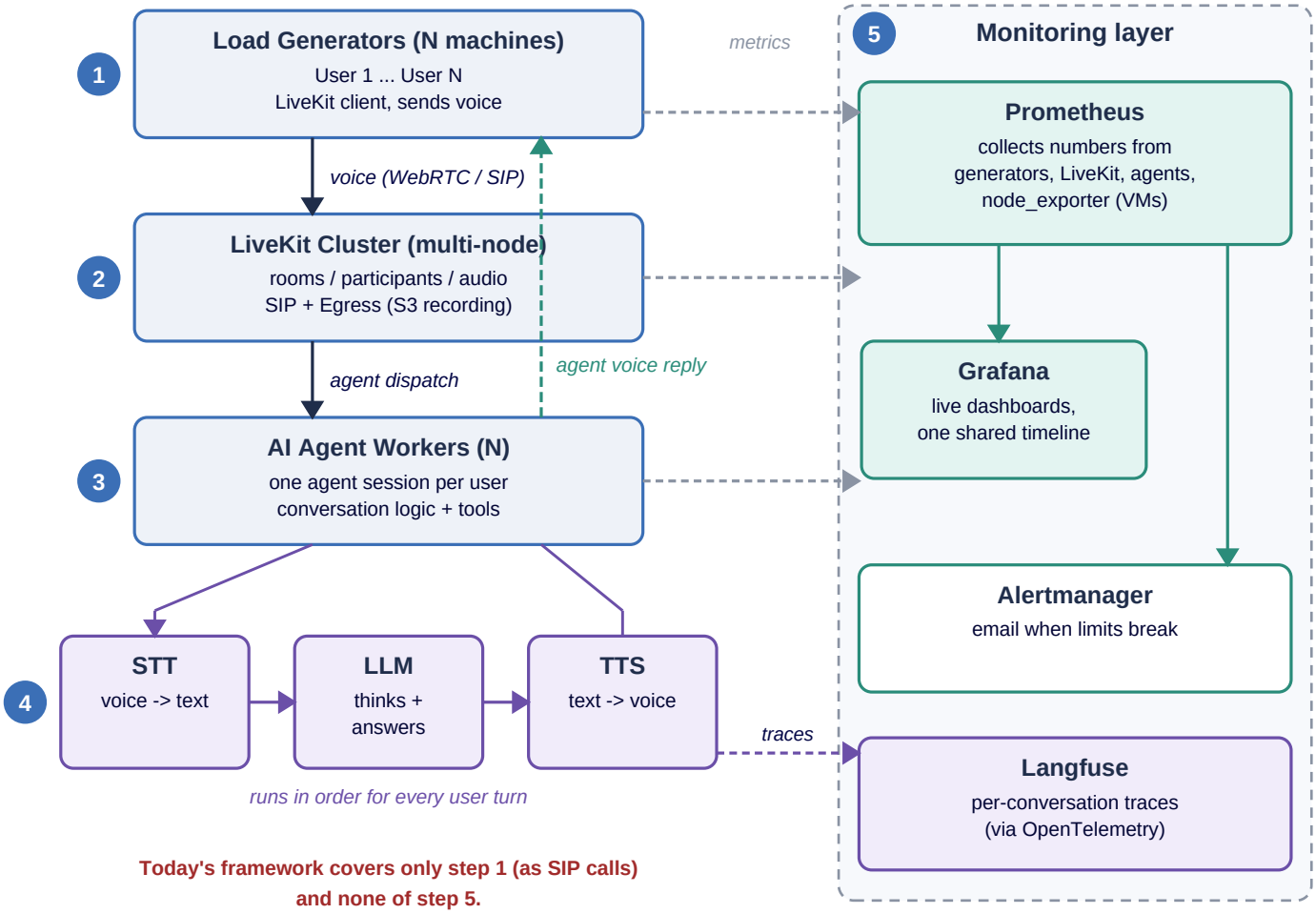


Connection in simple words

1. **Benchmark tool -> API server.** The benchmark sends many "make a call" requests at a chosen speed (steady, burst, or slowly increasing).
2. **API server -> LiveKit.** For each request the API creates a LiveKit room, dials a phone number through SIP, and plays a WAV file into the call.
3. **LiveKit -> SIP trunk.** LiveKit sends the call out through the SIP trunk: Twilio for a few real phones, or SIPp (a fake auto-answering phone) for big tests.
4. **Results.** The benchmark prints success rate, calls per second and API response time, plus CPU/RAM of the test machine, and can save them as JSON.

Key point for the team: today we measure how fast the API *accepts* a call. We do not yet measure whether the call connected, how fast the AI agent answered, or audio quality. One machine handles roughly 50-150 calls.

Diagram 2 - Target architecture (from team discussion)



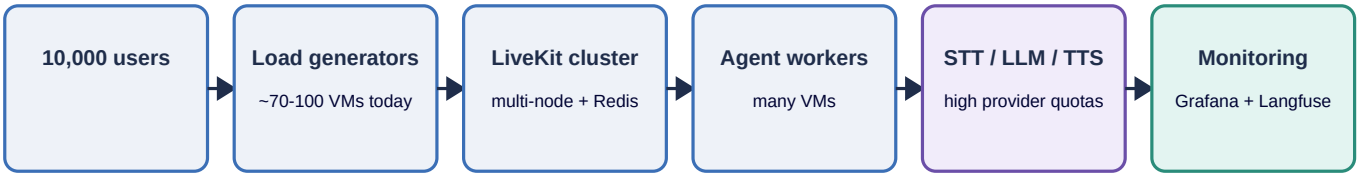
Connection in simple words

1. **Load generators -> LiveKit.** Generator machines create many simulated users. Each user joins a LiveKit room (WebRTC) or calls in (SIP) and speaks a recorded question.
2. **LiveKit manages the session.** LiveKit handles rooms, participants, audio, SIP and recording (Egress to S3).
3. **LiveKit -> AI agent.** LiveKit dispatches each user session to an AI agent worker.
4. **Agent pipeline.** The agent runs one step after another for every user turn: **User voice -> STT -> LLM -> TTS -> Agent voice -> User**.
5. **Monitoring.** Prometheus collects numbers from generators, LiveKit, agents and VMs. Grafana shows them on one live timeline, Alertmanager emails when limits are crossed, and Langfuse stores per-conversation traces.

From Diagram 1 to Diagram 2: what changes

Area	Today (Diagram 1)	Target (Diagram 2)
Simulated user	SIP call that plays a WAV	LiveKit client that speaks and waits for the agent's reply
Scale	1 machine, ~50-150 calls	Many generator machines, 100 -> 10,000 users
Main metric	API response time	Call setup time + agent response time (TTFT, TTS TTFB, end-to-end)
Where results go	Console + JSON at the end	Prometheus + Grafana live, alerts, Langfuse traces
Call endpoint	Twilio	SIPp mock for big tests, Twilio for a small real sample

Scaling to 10,000 users



Every box must scale out together: the slowest box sets the real user limit.

Before planning this tier, confirm whether 10,000 means **concurrent** users (very large) or users **per day** (usually a few hundred at once).

How to present it (about 5 minutes)

1. Goal (30 s)

> "We want to know how many users our voice agent can handle: 100, then 500, 1,000, 2,500 and 10,000, and what breaks first."

2. Diagram 1 (1 min)

> "This is what we have today. The benchmark fires call requests, our API turns each one into a LiveKit SIP call that plays a recording. It works and gives us success rate and API latency, for about 50-150 calls per machine."

3. The gap (1 min)

> "It only tells us the API accepted the call. It doesn't tell us whether the call connected or how fast the AI agent replied, and its numbers aren't in Grafana."

4. Diagram 2 (1.5 min)

> "This is the target. Simulated users talk to the agent through LiveKit. The agent runs STT, LLM, TTS. Prometheus collects numbers from every box, Grafana shows them on one timeline, Langfuse shows each conversation."

5. Plan (1 min)

> "We go tier by tier: fix and measure at 100, add more generators for 500, make the generator talk to the agent for 1,000, raise provider limits for 2,500, then size 10,000 from real numbers."

Questions to ask the team at the end

- Is 10,000 concurrent users or users per day?
- Do we test WebRTC users talking to the agent first, or SIP phone calls?
- Is our Twilio account trial or paid, and what are its call limits?
- Is LiveKit self-hosted (so Prometheus metrics are available) or LiveKit Cloud?